

Отчёт по лабораторной работе №8

Компьютерный практикум по статистическому анализу данных

Канева Екатерина, НФИбд-02-22

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическая часть	7
4	Выполнение лабораторной работы	8
4.1	Примеры	8
4.2	Задания для самостоятельной работы	12
4.2.1	Задание 1	13
4.2.2	Задание 2	13
4.2.3	Задание 3	14
4.2.4	Задание 4	15
4.2.5	Задание 5	16
5	Выводы	18

Список иллюстраций

4.1	Примеры с линейным программированием.	8
4.2	Примеры с векторизованными ограничениями и целевой функцией оптимизации.	9
4.3	Оптимизация рациона питания (1).	9
4.4	Оптимизация рациона питания (2).	10
4.5	Путешествие по миру.	10
4.6	Портфельные инвестиции (1).	11
4.7	Портфельные инвестиции (2).	11
4.8	Восстановление изображения (1).	12
4.9	Восстановление изображения (2).	12
4.10	Задание 1.	13
4.11	Задание 2.	14
4.12	Задание 3.	15
4.13	Задание 4.	16
4.14	Задание 5.	17

Список таблиц

1 Цель работы

Основная цель работы — освоить пакеты Julia для решения задач оптимизации.

2 Задание

- Используя Jupyter Lab, повторить примеры.
- Выполнить задания для самостоятельной работы.

3 Теоретическая часть

Julia - высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия.

Для выполнения заданий была использована официальная документация Julia.

4 Выполнение лабораторной работы

4.1 Примеры

Сначала я выполнила примеры из лабораторной работы (рис. 4.1-4.9):

```
[3]: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer);

[4]: # Определение переменных x, y и граничных условий для них:
@variable(model, x >= 0)
@variable(model, y >= 0);

[5]: # Определение ограничений модели:
@constraint(model, 6x + 8y >= 100)
@constraint(model, 7x + 12y >= 120);

[6]: # Определение целевой функции:
@objective(model, Min, 12x + 20y);

[7]: # Вызов функции оптимизации:
optimize!(model)

[8]: # Определение причины завершения работы оптимизатора:
termination_status(model)

[8]: OPTIMAL::TerminationStatusCode = 1

[9]: # Демонстрация первичных результирующих значений переменных x и y:
@show value(x);
@show value(y);
# Демонстрация результата оптимизации:
@show objective_value(model);

value(x) = 14.999999999999993
value(y) = 1.2500000000000047
objective_value(model) = 205.0
```

Рис. 4.1: Примеры с линейным программированием.

```

[19]: # Определение объекта модели с именем vector_model:
vector_model = Model(GLPK.Optimizer);

[21]: # Определение начальных данных:
A = [ 1 1 9 5;
      3 5 0 8;
      2 0 6 13];
b = [7; 3; 5];
c = [1; 3; 5; 2];

[23]: # Определение вектора переменных:
@variable(vector_model, x[1:4] >= 0);

[25]: # Определение ограничений модели:
@constraint(vector_model, A * x .== b);

[26]: # Определение целевой функции:
@objective(vector_model, Min, c' * x);

[29]: # Вызов функции оптимизации:
optimize!(vector_model)

[31]: # Определение причины завершения работы оптимизатора:
termination_status(vector_model)

[31]: OPTIMAL::TerminationStatusCode = 1

[33]: # Демонстрация результата оптимизации:
@show objective_value(vector_model);
objective_value(vector_model) = 4.9230769230769225

```

Рис. 4.2: Примеры с векторизованными ограничениями и целевой функцией оптимизации.

```

[36]: category_data = JuMP.Containers.DenseAxisArray(
      [1800 2200;
       91 Inf;
       0 65;
       0 1779],
      ["calories", "protein", "fat", "sodium"],
      ["min", "max"]);

[38]: # массив данных с наименованиями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"];

[40]: # Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray(
      [2.49, 2.89, 1.50, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59],
      foods);

[42]: food_data = JuMP.Containers.DenseAxisArray(
      [410 24 26 730;
       420 32 10 1190;
       560 20 32 1800;
       380 4 19 270;
       320 12 10 930;
       320 15 12 820;
       320 31 12 1230;
       100 8 2.5 125;
       330 8 10 180],
      foods,
      ["calories", "protein", "fat", "sodium"]);

[44]: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer);

[46]: # Определим массив:
categories = ["calories", "protein", "fat", "sodium"];

```

Рис. 4.3: Оптимизация рациона питания (1).

```
[48]: # Определение переменных:
@variables(model, begin
    category_data[c, "min"] <= nutrition[c = categories] <= category_data[c, "max"]
    # Сколько покупать продуктов:
    buy[foods] >= 0
end);

[50]: # Определение целевой функции:
@objective(model, Min, sum(cost[f] * buy[f] for f in foods));

[52]: # Определение ограничений модели:
@constraint(model, [c in categories],
    sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c]);

[54]: # Вызов функции оптимизации:
JuMP.optimize!(model)
term_status = JuMP.termination_status(model)

[54]: OPTIMAL::TerminationStatusCode = 1

[56]: hcat(buy.data, JuMP.value.(buy.data))

[56]: 9x2 Matrix{AffExpr}:
buy[hamburger]  0.6045138888888888
buy[chicken]    0
buy[hot dog]    0
buy[fries]      0
buy[macaroni]   0
buy[pizza]      0
buy[salad]      0
buy[milk]       6.9701388888888935
buy[ice cream]  2.5913194444444441
```

Рис. 4.4: Оптимизация рациона питания (2).

```
[61]: # Подключение пакетов:
import Pkg
Pkg.add("DelimitedFiles")
Pkg.add("CSV")
using DelimitedFiles
using CSV;

Resolving package versions...
Project No packages added to or removed from `C:\Users\ekaneva\.julia\environments\v1.12\Project.toml`
Manifest No packages added to or removed from `C:\Users\ekaneva\.julia\environments\v1.12\Manifest.toml`
Resolving package versions...
Project No packages added to or removed from `C:\Users\ekaneva\.julia\environments\v1.12\Project.toml`
Manifest No packages added to or removed from `C:\Users\ekaneva\.julia\environments\v1.12\Manifest.toml`

[62]: # Считывание данных:
passportdata = readlm(joinpath("passport-index-dataset", "passport-index-matrix.csv"), ',');

[63]: # Задаём переменные:
cntr = passportdata[2:end, 1]
vf = (x -> typeof(x) == Int64 || x == "VF" || x == "VOA" ? 1 : 0).(passportdata[2:end, 2:end]);

[64]: # Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}();

[65]: # Переменные, ограничения и целевая функция:
@variable(model, pass[1:length(cntr)], Bin)
@constraint(model, [j=1:length(cntr)], sum( vf[i,j]*pass[i] for i in 1:length(cntr)) >= 1)
@objective(model, Min, sum(pass));

[66]: # Вызов функции оптимизации:
JuMP.optimize!(model)
termination_status(model)

[66]: OPTIMAL::TerminationStatusCode = 1

[73]: # Просмотр результата:
print(JuMP.objective_value(model), " passports: ", join(cntr[findall(JuMP.value.(pass) .== 1)], ", "))

34.0 passports: Afghanistan, Australia, Bahrain, Cameroon, Comoros, Congo, Djibouti, Dominican Republic, Eritrea, Guinea-Bissau, Hong Kong, Iran, Kenya, Kuwait, Liberia, Libya, Madagascar, Maldives, Mauritania, Morocco, Nepal, New Zealand, North Korea, Palestine, Papua New Guinea, Qatar, Saudi Arabia, Singapore, Somalia, South Sudan, Spain, Syria, Turkmenistan, United States
```

Рис. 4.5: Путешествие по миру.

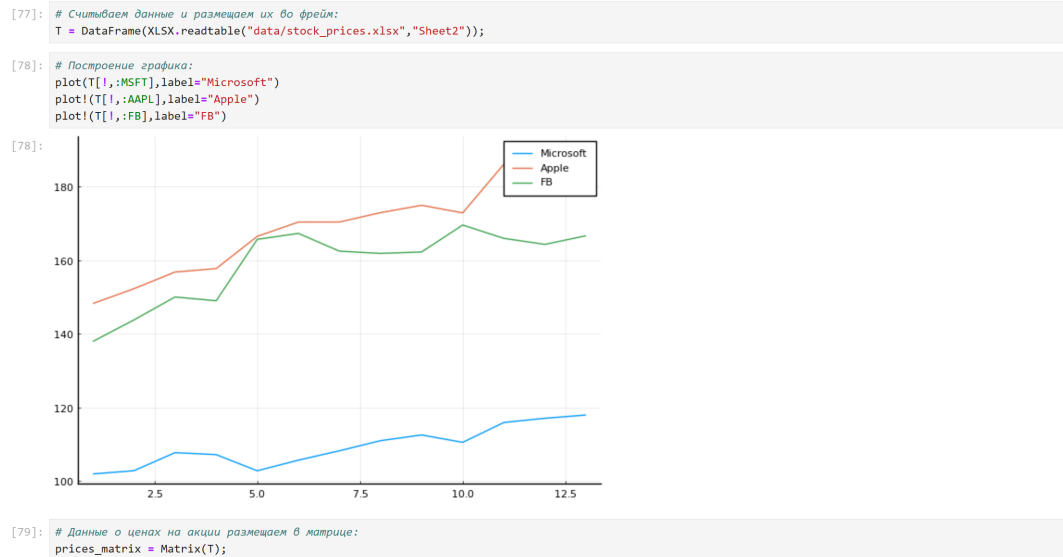


Рис. 4.6: Портфельные инвестиции (1).

```
-----
objective = 0.000462
-----
```

```
[96]: x
```

```
[96]: Variable
size: (3, 1)
sign: real
convexity: affine
id: 160.099
value: [0.07740709795031023, 0.11606771003983785, 0.8065276266877958]
```

```
[98]: sum(x.value)
```

```
[98]: 1.000002434677944
```

```
[100]: r*x.value
```

```
[100]: 1x1 adjoint(::Vector{Float64}) with eltype Float64:
0.019947233108531883
```

```
[101]: x.value .* 1000
```

```
[101]: 3x1 Matrix{Float64}:
77.40709795031023
116.06771003983785
806.5276266877958
```

Рис. 4.7: Портфельные инвестиции (2).

```
[107]: # Считывание исходного изображения:
Kref = load("data/khiam-small.jpg");

[109]: K = copy(Kref)
p = prod(size(K))
missingids = rand(1:p,400)
K[missingids] .= RGBX{N0f8}(0.0,0.0,0.0)
K
Gray.(K)

[109]: 
```

```
[110]: # Матрица цветов:
Y = Float64.(Gray.(K));

[111]: correctids = findall(Y[:,1]≠0)
X = Convex.Variable(size(Y))
problem = minimize(nuclearnorm(X))
problem.constraints += X[correctids]==Y[correctids];

Warning: Concatenating collections of constraints together with `+` or `+=` to produce a new list of constraints is deprecated. Instead, use `vcat` to concatenate collections of constraints.
@ Convex C:\Users\ekaneval\julia\packages\Convex\IPPoR\src\deprecations.jl:129

[115]: # Находим решение:
solve!(problem, SCS.Optimizer);
```

Рис. 4.8: Восстановление изображения (1).

```
[116]: @show norm(float.(Gray.(Kref))-X.value)
@show norm(-X.value)
colorview(Gray, X.value)

norm(float.(Gray.(Kref)) - X.value) = 1.1938997182548805
norm(-X.value) = 124.33410402043593

[116]: 
```

Рис. 4.9: Восстановление изображения (2).

4.2 Задания для самостоятельной работы

Далее я приступила к выполнению заданий для самостоятельной работы.

4.2.1 Задание 1

Необходимо было решить задачу линейного программирования при заданных ограничениях. Решила задачу аналогично первому примеру из лабораторной (рис. 4.10):

```
[120]: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer);

[121]: # Определение переменных x, y и граничных условий для них:
@variable(model, 0 <= x1 <= 10)
@variable(model, x2 >= 0)
@variable(model, x3 >= 0);

[123]: # Определение ограничений модели:
@constraint(model, -x1 + x2 + 3x3 <= -5)
@constraint(model, x1 + 3x2 - 7x3 <= 10);

[124]: # Определение целевой функции:
@objective(model, Max, x1 + 2x2 + 5x3);

[125]: # Вызов функции оптимизации:
optimize!(model)
# Определение причины завершения работы оптимизатора:
termination_status(model)

[125]: OPTIMAL::TerminationStatusCode = 1

[130]: @show value(x1);
@show value(x2);
@show value(x3);

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375
```

Рис. 4.10: Задание 1.

4.2.2 Задание 2

Далее необходимо было решить ту же задачу линейного программирования при заданных ограничениях, как в задании 1, но с массивами. Решила задачу аналогично второму примеру из лабораторной (рис. 4.11):

```

[133]: # Определение объекта модели с именем model:
vector_model = Model(GLPK.Optimizer);

[135]: A = [-1 1 3; 1 3 -7]
      b = [-5; 10]
      c = [1; 2; 5];

[137]: @variable(vector_model, x[1:3] >= 0);

[139]: # Определение ограничений модели:
      @constraint(vector_model, A * x .<= b)
      @constraint(vector_model, x[1] <= 10);

[141]: # Определение целевой функции:
      @objective(vector_model, Max, c' * x);

[143]: # Вызов функции оптимизации:
      optimize!(vector_model)
      # Определение причины завершения работы оптимизатора:
      termination_status(vector_model)

[143]: OPTIMAL::TerminationStatusCode = 1

[145]: # Демонстрация первичных результирующих значений переменных x и y:
      @show value(x[1]);
      @show value(x[2]);
      @show value(x[3]);

      value(x[1]) = 10.0
      value(x[2]) = 2.1875
      value(x[3]) = 0.9375

[147]: # Демонстрация результата оптимизации:
      @show objective_value(vector_model);

      objective_value(vector_model) = 19.0625

```

Рис. 4.11: Задание 2.

4.2.3 Задание 3

Необходимо было решить задачу выпуклого программирования - решить задачу оптимизации. Решила задачу аналогично примерам из первой части лабораторной (рис. 4.12):

```
[149]: using Random

[152]: m = 8
       n = 4
       Random.seed!(1)
       A = rand(m, n)
       b = rand(m);

[154]: x = Variable(n);

[156]: objective = sumsquares(A*x - b)
       constraints = [x >= 0];

[158]: problem = minimize(objective, constraints);

[159]: solve!(problem, SCS.Optimizer);

[ Info: [Convex.jl] Compilation finished: 5.27 seconds, 234.278 MiB of memory allocated
-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 5, constraints m: 14
cones:   1: linear vars: 4
        q: soc vars: 10, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys: sparse-direct-amd-qdldl
          nnz(A): 38, nnz(P): 0
-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
  0 | 1.66e+01 | 1.00e+00 | 2.31e+01 | -1.14e+01 | 1.00e-01 | 7.84e-04
125 | 5.09e-06 | 2.11e-06 | 1.38e-05 | 6.66e-01 | 1.23e+00 | 2.22e-03
-----
status: solved
timings: total: 2.23e-03s = setup: 1.72e-04s + solve: 2.05e-03s
          lin-sys: 7.87e-05s, cones: 3.78e-05s, accel: 1.08e-05s
-----
objective = 0.665534
```

Рис. 4.12: Задание 3.

4.2.4 Задание 4

Необходимо было решить задачу оптимальной раскладки по залам. Решила задачу аналогично примерам из первой части лабораторной (рис. 4.13):

```

[163]: num_sections = 5
       num_applications = 1000

       min_capacity = [180, 180, 220, 180, 180]
       max_capacity = [250, 250, 220, 250, 250]

       Random.seed!(2)
       priorities = [shuffle([1, 2, 3, 10000, 10000]) for _ in 1:num_applications]
       priorities = hcat(priorities...)
       priority_weights = sum(priorities, dims=1);

[164]: model = Model(GLPK.Optimizer);

[165]: @variables(model, begin
       180 <= x1 <= 250
       180 <= x2 <= 250
       x3 == 220
       180 <= x4 <= 250
       180 <= x5 <= 250
       end);

[166]: @constraint(model, x1 + x2 + x3 + x4 + x5 == 1000)
       @objective(model, Min, x1 * priority_weights[1] +
       x2 * priority_weights[2] +
       x3 * priority_weights[3] +
       x4 * priority_weights[4] +
       x5 * priority_weights[5]);

[167]: optimize!(model)
       termination_status(model)

[167]: OPTIMAL::TerminationStatusCode = 1

[168]: println("Результаты:")
       println("x1: ", value(x1))
       println("x2: ", value(x2))
       println("x3: ", value(x3))
       println("x4: ", value(x4))
       println("x5: ", value(x5))
       println("Оптимальное значение целевой функции: ", objective_value(model))

Результаты:
x1: 180.0
x2: 180.0
x3: 220.0
x4: 240.0
x5: 180.0
Оптимальное значение целевой функции: 3.99660192e9

```

Рис. 4.13: Задание 4.

4.2.5 Задание 5

Необходимо было решить задачу о плане приготовления кофе. Решила задачу аналогично примеру о портфельных инвестициях из лабораторной (рис. 4.14):

```

[176]: coffee = ["Raf", "Cappuccino"]
       products = ["grains", "milk", "sugar"]

       cost = JuMP.Containers.DenseAxisArray(
           [400, 300],
           coffee
       )

       coffee_data = JuMP.Containers.DenseAxisArray(
           [40 140 5;
            30 120 0],
           coffee,
           products
       )

       store = JuMP.Containers.DenseAxisArray([500, 2000, 40], products);

[178]: model = Model(GLPK.Optimizer);

[180]: @variables(model, begin
       buy[coffee] >= 0
       end)
       @objective(model, Max, sum(cost[i] * buy[i] for i in coffee))
       @constraint(model, [p in products], sum(coffee_data[i, p] * buy[i] for i in coffee) <= store[p])
       @constraint(model, coffee_data["Raf", "sugar"] * buy["Raf"] == 40);

[181]: JuMP.optimize!(model)
       term_status = JuMP.termination_status(model)

[181]: OPTIMAL::TerminationStatusCode = 1

[184]: hcat(buy.data, JuMP.value.(buy.data))

[184]: 2x2 Matrix{AffExpr}:
       buy[Raf]      8
       buy[Cappuccino] 6

```

Рис. 4.14: Задание 5.

5 Выводы

Освоила пакеты Julia для решения задач оптимизации.